

Skill Week-5 (GET & POST Methods)

GET API - Retrieve all students

```
@app.get("/students")
def get_students():

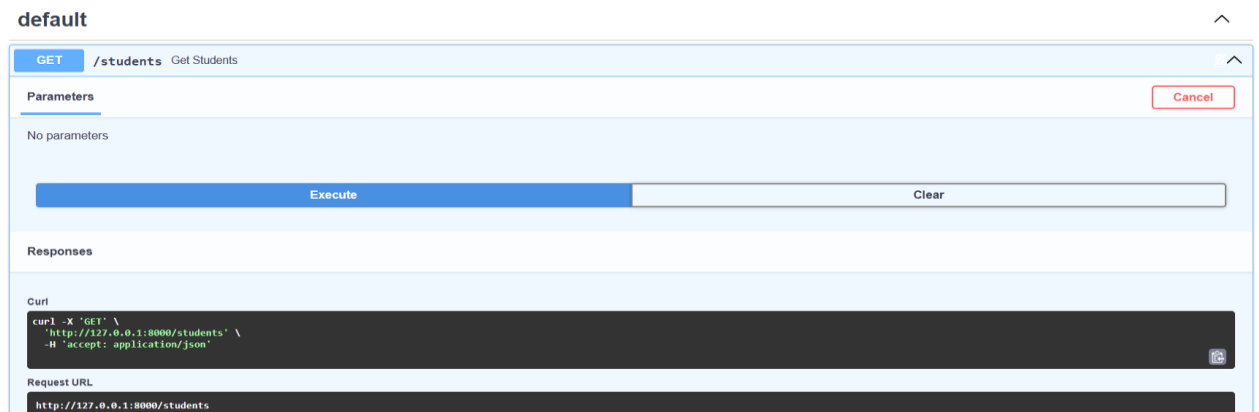
    with engine.connect() as connection:
        result = connection.execute(
            text("SELECT * FROM students")
        )

        students = result.mappings().all()

    return {
        "message": "Students retrieved successfully",
        "students": students
    }
```

What is shown on Swagger for above code-

FastAPI 0.1.0 OAS 3.1
/openapi.json



Server response

Code	Details
200	Response body <pre>{ "message": "Students retrieved successfully", "students": [{ "id": 0, "name": "string", "department": "string" }, { "id": 101, "name": "Ravi", "department": "CSE" }, { "id": 102, "name": "Priya", "department": "CSE" }] }</pre> Response headers <pre>content-length: 194 content-type: application/json date: Wed, 09 Sep 2026 09:50:04 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type:

Controls Accept header.

Example Value | Schema

```
"string"
```

POST API - Add a new student

```
@app.post("/students")
def create_student(student: Student):

    with engine.begin() as connection:
        connection.execute(
            text("""
                INSERT INTO students (id, name, department)
                VALUES (:id, :name, :department)
            """),
            {
                "id": student.id,
                "name": student.name,
                "department": student.department
            }
        )
```

What is shown for above POST code on Swagger-

POST /students Create Student

Parameters

No parameters

Request body **required**

Edit Value | Schema

```
{
  "id": 103,
  "name": "Ananya",
  "department": "CSE"
}
```

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/students' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 103,
    "name": "Ananya",
    "department": "CSE"
  }'
```

Request URL

http://127.0.0.1:8000/students

Server response

Code	Details
------	---------

200

Response body

```
{
  "message": "Student created successfully",
  "student": {
    "id": 103,
    "name": "Ananya",
    "department": "CSE"
  }
}
```

Response headers

```
content-length: 98
content-type: application/json
date: Wed, 09 Sep 2026 09:52:16 GMT
server: uvicorn
```

Before Posting Student Data:

Pretty print ☒

```
{
  "message": "Students retrieved successfully",
  "students": [
    {
      "id": 0,
      "name": "string",
      "department": "string"
    },
    {
      "id": 101,
      "name": "Ravi",
      "department": "CSE"
    },
    {
      "id": 102,
      "name": "Priya",
      "department": "CSE"
    },
    {
      "id": 103,
      "name": "Ananya",
      "department": "CSE"
    }
  ]
}
```

After posting student-104:

Pretty print ☒

```
{
  "message": "Students retrieved successfully",
  "students": [
    {
      "id": 0,
      "name": "string",
      "department": "string"
    },
    {
      "id": 101,
      "name": "Ravi",
      "department": "CSE"
    },
    {
      "id": 102,
      "name": "Priya",
      "department": "CSE"
    },
    {
      "id": 103,
      "name": "Ananya",
      "department": "CSE"
    },
    {
      "id": 104,
      "name": "Lavanya",
      "department": "CS&IT"
    }
  ]
}
```

PUT - Update an existing student

```
@app.put("/students/{student_id}")
def update_student(student_id: int, student: Student):

    with engine.begin() as connection:

        result = connection.execute(
            text("""
                UPDATE students
                SET name = :name,
                    department = :department
                WHERE id = :student_id
            """),
            {
                "student_id": student_id,
                "name": student.name,
                "department": student.department
            }
        )

    if result.rowcount == 0:
        return {
            "message": "Student not found"
        }

    return {
        "message": "Student updated successfully",
        "student": student
    }
```

PUT

/students/{student_id} Update Student

^

Parameters

Cancel

Reset

Name	Description
student_id * required	
integer	104
(path)	

Request body required

application/json

Edit Value | Schema

```
{
  "id": 104,
  "name": "Priya",
  "department": "CSE"
}
```

Responses

Curl

```
curl -X 'PUT' \
  http://127.0.0.1:8000/students/104 \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 104,
    "name": "Priya",
    "department": "CSE"
  }'
```

Request URL

http://127.0.0.1:8000/students/104

Output after editing using PUT:

```
{
  "id": 104,
  "name": "Priya",
  "department": "CSE"
}
```

DELETE - Delete a student

```
@app.delete("/students/{student_id}")
```

```
def delete_student(student_id: int):
```

```
    with engine.begin() as connection:
```

```
        result = connection.execute(
            text("""
                DELETE FROM students
                WHERE id = :student_id
            """),
            {
                "student_id": student_id
            }
        )
```

```
    if result.rowcount == 0:
```

```
        return {
            "message": "Student not found"
        }
```

Output:

DELETE /students/{student_id} Delete Student	
Parameters	
Name	Description
student_id * required	
integer (path)	103
Execute	
Responses	
Curl	
curl -X 'DELETE' \	
http://127.0.0.1:8000/students/103' \	
-H 'accept: application/json'	
Request URL	
http://127.0.0.1:8000/students/103	
Server response	
Code	Details
200	Response body
{	
"message": "Student deleted successfully"	
}	

Pretty print ☒

```
{
  "message": "Students retrieved successfully",
  "students": [
    {
      "id": 0,
      "name": "string",
      "department": "string"
    },
    {
      "id": 101,
      "name": "Ravi",
      "department": "CSE"
    },
    {
      "id": 102,
      "name": "Priya",
      "department": "CSE"
    },
    {
      "id": 104,
      "name": "Priya",
      "department": "CSE"
    }
  ]
}
```

SQL Workbench Code:

```
CREATE DATABASE university;
```

```
USE university;
```

```
CREATE TABLE students (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  department VARCHAR(100)
);
```

```
SHOW DATABASES;
```

```
USE university;
```

```
INSERT INTO students (id, name, department)
```

```
VALUES
```

```
(101, 'Ravi', 'CSE'),
```

```
(102, 'Priya', 'CSE');
```

```
SHOW TABLES;
```

```
SELECT * FROM students;
```

Workbench Output:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	id	name	department
▶	101	Ravi	CSE
	102	Priya	CSE
*	NULL	NULL	NULL

students 3 x Apply Revert

Output

Action Output

#	Time	Action	Message
✓ 4	15:17:05	CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(100), ...	0 row(s) affected
✓ 5	15:17:12	SHOW DATABASES	15 row(s) returned
✓ 6	15:17:33	USE university	0 row(s) affected
✓ 7	15:17:33	SHOW TABLES	1 row(s) returned
✓ 8	15:17:46	INSERT INTO students (id, name, department) VALUES (101, 'Ravi', 'CSE'), (102, '...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
✓ 9	15:17:53	SELECT * FROM students LIMIT 0, 1000	2 row(s) returned